

Key-Value Store v2

Persistenza locale, crash recovery e coerenza interna

Architetture dei Sistemi Distribuiti

Lezione 11

Il percorso fin qui:

- v0: nodo singolo e contratto dell'interfaccia;
- v1: concorrenza, lock, safety e liveness;
- v2: persistenza locale e crash recovery.

La domanda nuova e': quando una scrittura puo' essere considerata davvero confermata?

Da questa tappa distinguiamo due stati:

- stato attivo: il dizionario in RAM;
- stato ricostruibile: cio' che possiamo recuperare da disco dopo un crash.

Il problema nasce quando i due non evolvono con lo stesso ordine logico.

Sequenza della variante con snapshot periodico:

- 1 aggiornare la RAM;
- 2 rispondere OK;
- 3 salvare piu' tardi uno snapshot.

Vantaggio: path rapido.

Rischio: una scrittura ackata puo' sparire dopo crash.

Path unsafe nel codice

```
with self._lock:  
    self._data[key] = value  
    self._dirty = True  
return "OK", False
```

Lo snapshot e' delegato a un thread separato:

```
while not self._stop_event.wait(2.0):  
    self.flush_snapshot()
```

Sequenza astratta:

- AcquireLock -> ApplyInMemory -> Reply -> BackgroundSnapshot

Finestra critica:

- crash dopo Reply;
- snapshot non ancora eseguito;
- recovery che perde una scrittura già ackata.

Sequenza della variante con write-ahead log:

- 1 serializzare l'intento;
- 2 appendere il record al log;
- 3 eseguire `fsync`;
- 4 applicare l'update in RAM;
- 5 rispondere OK.

```
with self._lock:
    self._append_record({
        "op": "SET",
        "key": key,
        "value": value,
    })
    self._data[key] = value
return "OK", False
```

Il punto chiave e' che `_append_record` esegue anche `fsync`.

Il write-ahead log difende questo invariante:

Invariante

Ogni scrittura che ha gia' ricevuto OK deve essere ricostruibile dopo crash.

Il disco non deve necessariamente rispecchiare in ogni istante tutta la RAM. Deve pero' contenere abbastanza informazione per ricostruire le scritture promesse.

Sequenza astratta:

- AcquireLock -> AppendLog -> Fsync -> ApplyInMemory -> Reply

Osservazioni:

- prima di Fsync non si puo' ancora difendere OK;
- tra Fsync e ApplyInMemory il processo vivo puo' essere incompleto;
- il replay del log chiude il gap al riavvio.

Esempi di safety in questa tappa:

- nessuna scrittura ackata viene dimenticata dalla recovery;
- il replay non produce uno stato impossibile;
- DELETE e INCR restano coerenti rispetto al log.

Il costo introdotto dalla durabilit :

- `fsync` allunga la latenza delle scritture;
- trattenere il lock durante persistenza riduce il parallelismo;
- un disco lento puo' rallentare l'intero servizio.

Anche qui safety e liveness vanno discusse insieme.

All'avvio la variante safe rilegge il log:

```
for raw_line in log_file:
    record = json.loads(raw_line)
    self._apply_record(record)
```

Una coda finale troncata del file puo' essere ignorata se non e' un record JSON completo.

Variante unsafe:

- 1 SET course distributed-systems
- 2 CRASH
- 3 restart
- 4 GET course

Variante safe:

- 1 stessa sequenza;
- 2 esito atteso differente dopo restart.

La stessa API puo' restare stabile, ma il suo contratto cambia.

In questa tappa OK non significa solo:

- update coerente in RAM.

Significa anche:

- update ricostruibile dopo crash locale.

Lavoro richiesto:

- ① analizzare la variante snapshot unsafe;
- ② identificare la finestra di crash;
- ③ confrontarla con la variante write-ahead log;
- ④ discutere il costo di `fsync` su liveness e throughput.